

// **WAYSCONF 2026 · MASTERCLASS · 90 MINUTES**

Build an AI that checks and fixes its own work

Agentic loops in practice.

Szymon Paluch · CTO, Susteen

ROOM-PM · 16 September 2026

// 0:00 – 0:08

The case

Most AI demos stop here

A prompt goes in. Something impressive comes out. Everyone claps.

Then someone asks a question the demo was not built to survive:

Does it match the design?

Is the contrast legal?

Does it work on a phone?

Is that the actual copy, or copy it invented?

no answer

This is a talk about the word *no*

A loop is only worth building if something in it can refuse
the work without asking a language model for an opinion.

What you will have at 16:25

A landing page for a music festival that does not exist, built from a Figma design by an agent you instructed, checked by a verifier that does not care what the agent thinks, fixed by the agent when it failed, and deployed to a URL you can send to someone.

all gates green

```
npm run check
```



✓ 10/10 gates passed.

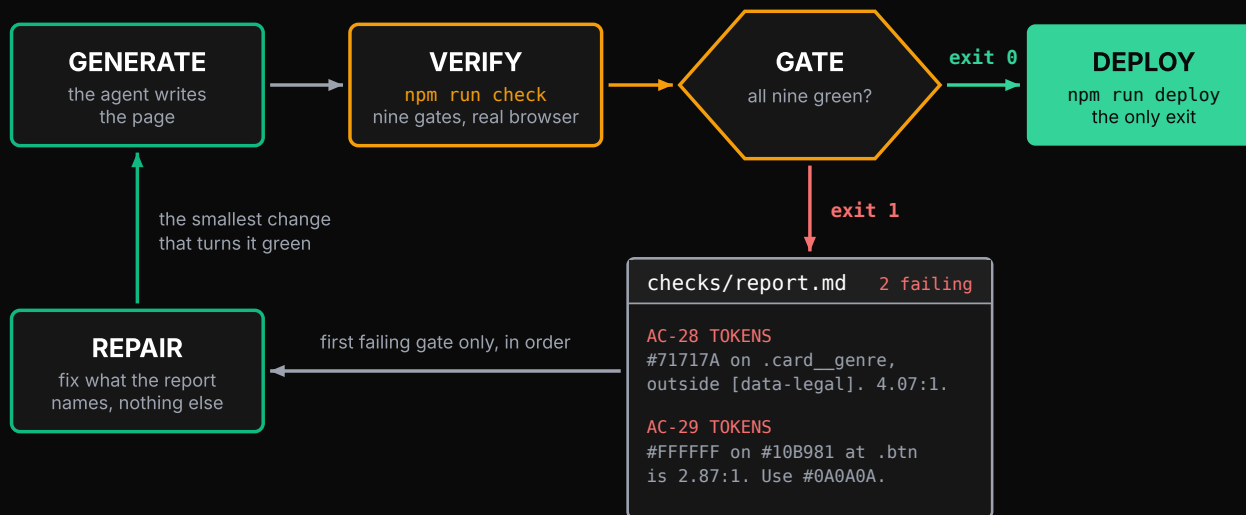
Build	PASS
Runtime	PASS
Structure	PASS
Accessibility	PASS
Design tokens	PASS
Visual fidelity	PASS
Content	PASS
Links	PASS
Text over images	PASS
Lighthouse	PASS



The loop

The loop

Four steps and one exit. The step that decides is a program, and it decides the same way every time.



The arrow back carries a file. Nothing on this path asks the model whether the work is good.

Checkpoint o

Everyone gets a working environment

You need two things

THE DESIGN

The TURBINE file in Figma, and a **design pack** — the same design as three PNGs, every colour and size, and every word.

THE PROMPTS

waysconf.szymonpaluch.com

Eight of them, with a copy button on each.

AND AN EMPTY FOLDER

```
mkdir turbine && cd turbine  
claude          # or: codex
```



There is **nothing to clone** and no code to read.

The project, the page, and the program that checks the page — the agent makes all three for itself.

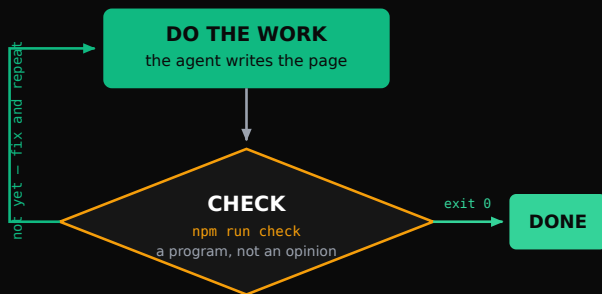
Two shapes, and neither is a script

Two shapes

Almost everything you will build on an agent is one of these. Neither is a script.

THE LOOP

do this again until a condition holds



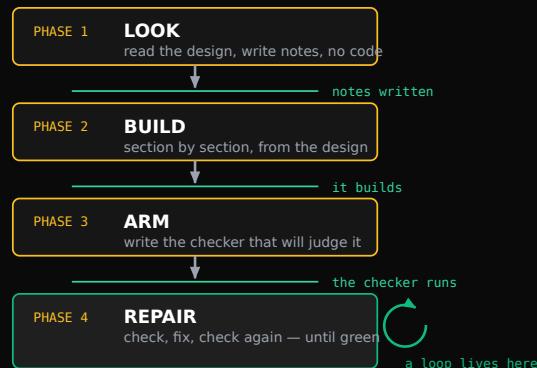
You define the **exit condition**.

"Run the check. Fix what it names. Repeat until it exits 0."

prompt 05 · /goal in Claude Code

THE WORKFLOW

do these, in order, with a bar between each



You define the **phases**, and what must be true before each one ends.

prompt 08

Neither of these is a script. Both are things you **say** — which is why changing one is a sentence, not an edit.

They nest: the workflow gets you to nearly-right, and a loop inside one phase closes the last gap.

The terminal, in one slide

```
pwd          # where am I
ls           # what is here
cd my-turbine # go in there
npm run dev  # start the site
```



Ctrl+C stops whatever is running.

It is a text field that runs programs.

It gives you no feedback while it works, which reads as "broken" and is almost always "busy".

When a command finishes you get the prompt back. When you do not get the prompt back, it is still going.

Full version: docs/TERMINAL-IN-TEN-MINUTES.md

// 0:15 — 0:25

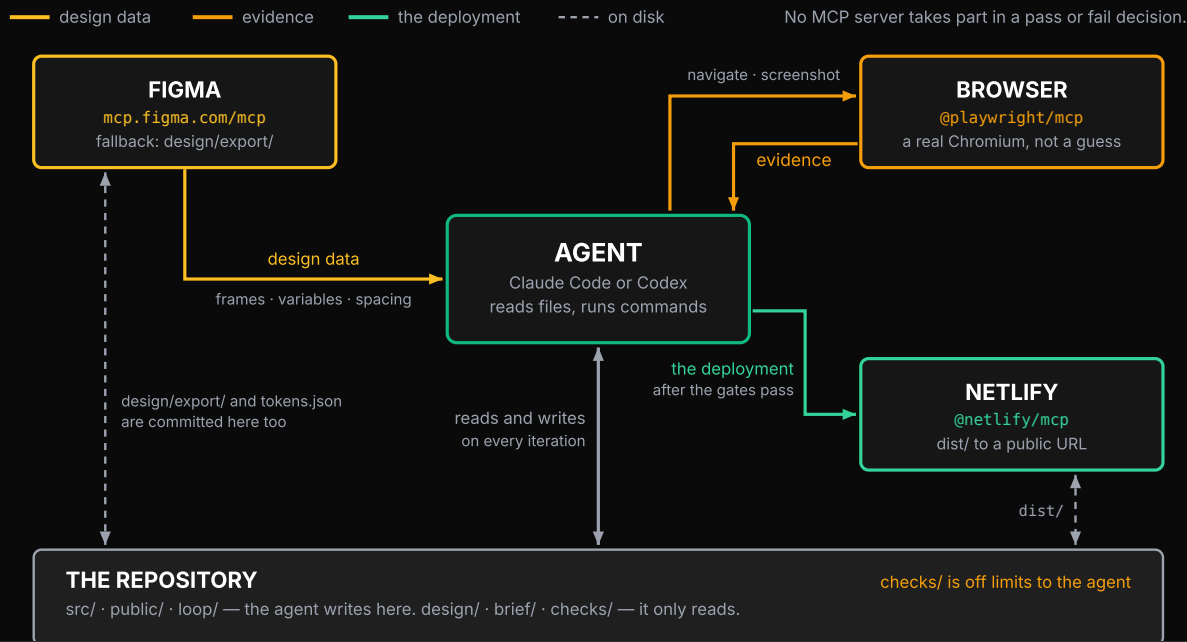
Design in, not screenshots in

MCP, in one sentence

A standard way for a coding agent to call tools that are not inside it.

What MCP actually connects

Three tools around one agent, and a repository underneath that every one of them reads and writes.



Two ways to hand over a design

A SCREENSHOT

The agent looks at pixels and infers.

The orange becomes *an* orange.

The spacing becomes *some* spacing.

It looks roughly right.

token gate fails

THE DESIGN ITSELF

Colours arrive as values. Spacing arrives as numbers.

```
"color": {  
  "accent": {  
    "sodium": { "$value": "#FF6A1A" }  
  }  
}
```



Nothing is guessed, so nothing is guessed wrong.

token gate passes

The honest bit about Figma

The official Figma MCP server needs a paid seat. On a free account it connects and then declines to hand anything over.

SO THE REPOSITORY SHIPS THE DESIGN AS DATA

```
design/export/    PNGs at 390, 768, 1440
design/tokens/    every colour, size, space
design/FIGMA-SPEC.md  the layout, in numbers
```



Nothing in this workshop is blocked by a free Figma account.

AND AS A PLUGIN THAT REBUILDS THE FILE

```
figma-plugin/
```



Run it inside Figma and it constructs the whole TURBINE file — variables, text styles, components, every frame — from the same tokens the code uses.

Plan first, then build

Read the plan before it writes the code

Plan mode is the cheapest correction in the entire loop. A wrong assumption costs one sentence to fix here, and twenty minutes of generated code to fix later.

```
# Claude Code: Shift+Tab until the footer says "plan mode on"  
# Codex CLI: ask for a plan and refuse to accept code until you have read it
```



Then read it. Actually read it. The plan is where you find out it misunderstood the brief.

Your turn — 15 minutes

```
mkdir turbine && cd turbine  
claude          # or: codex
```



Then, from the prompts page:

01 start → **02** look at the design → **03** build

Prompt 02 writes nothing. Read what it says it found before you let it near a file.

Falling behind cannot hurt you.

Stop where you are. Leave whatever is unfinished. I want to move on.

Every prompt stands alone. A half-built page with a working checker teaches more than a finished page with none.

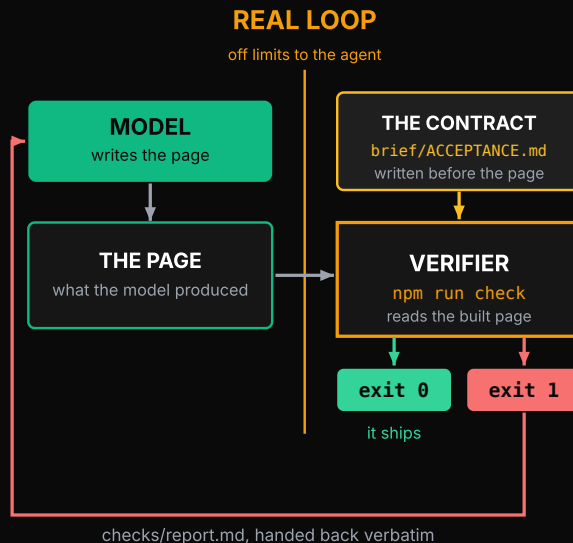
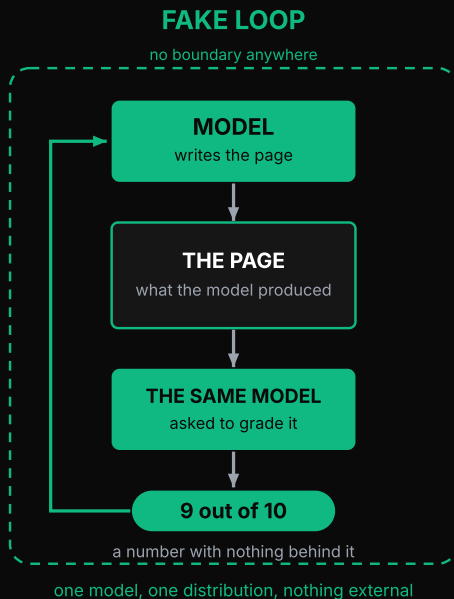
The verifier

The only part that matters

A real loop and a fake one

Fake loop, real loop

The same model writes the same page. What changes is who is allowed to say no.



Where self-assessment is fine, and where it is not

FINE

Is this sentence any good.

Which of these five findings matters most.

Does this read as the same brand.

Is the hierarchy clear.

Taste, prioritisation, judgement.

NOT FINE

Is this correct.

Is it accessible.

Does it match the design.

Is it finished.

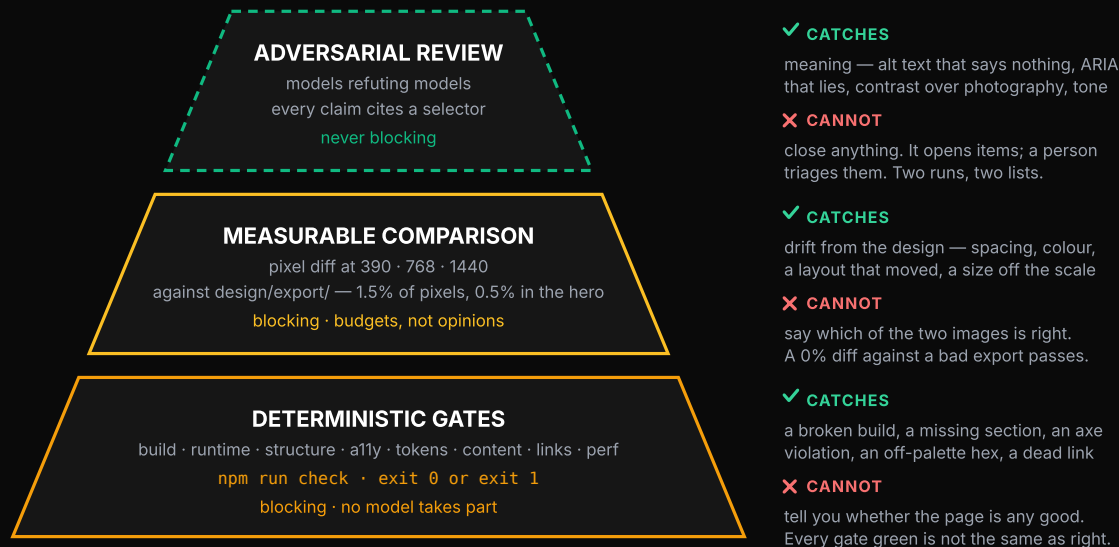
Anything with an external truth.

Use the external truth.

Three tiers of checking

Three tiers of verification

The bottom tier is the widest, the cheapest and the only kind that can be trusted to say no on its own.



The build gate runs first and alone; the seven browser gates then run together, and Lighthouse last.

Only the bottom two tiers can fail the build. Adversarial review runs after npm run check exits 0, and never blocks.

Automated accessibility rules reach roughly 30 to 40 per cent of WCAG — a widely cited estimate, not a measurement of this page.

Ten gates, sixty-one criteria, no opinions

GATE	WHAT IT DECIDES	CRITERIA
Build	It compiles, no type errors, one page out	AC-01...05
Runtime	No console errors, no failed requests	AC-03...04
Structure	Sections, headings, landmarks, skip link, ids	AC-06...14
Accessibility	axe clean at three widths, focus, keyboard, motion	AC-15...25
Design tokens	Every colour and size came from the design	AC-26...33
Visual fidelity	Pixels match the design, within a stated budget	AC-34...37
Content	The real copy, all twelve artists, no placeholders	AC-38...47
Links	Nothing points at nothing	AC-48...51
Lighthouse	Performance, a11y, best practices, SEO, and five budgets	AC-52...56
Text over images	The contrast axe refuses to judge	AC-61

Watch it catch something real

The palette has a trap in it, on purpose.

```
color.text.muted    #6B7280  
color.bg.base       #0A0B0D
```



4.07 : 1

Under the 4.5:1 that body text requires.

```
color.text.secondary #A7AEBB  
color.bg.base        #0A0B0D
```



8.9 : 1

Ask any model for "a muted grey for supporting copy" and watch which one it reaches for.

The failure report is the interface

```
**3. [AC-28] color.text.muted (#6B7280) used outside the footer legal block**
```

- Where: ``section[data-section="lineup"] .artist-card p``
- Expected: ``color.text.secondary (#A7AEBB)``
- Actual: ``#6B7280`` – 4.07:1 against #0A0B0D, minimum 4.5:1
- Hint: supporting copy uses text.secondary; muted is legal text only



No model wrote that. A program measured it and printed it.

The agent's next iteration starts by reading this file.

The rule that holds it all up

The agent may never edit the verifier.

It is written into `AGENTS.md`. It is checked in the dry-run harness. If an agent modifies anything under `checks/`, the run is recorded as compromised regardless of what the gates then say.

```
| Verifier modified by the agent | no |
```



Here is what actually happened

These prompts, an **empty folder** outside every repository, Codex. Exactly what you have in front of you now.

CODEx CLI, EMPTY FOLDER

01 — a project exists	107s
03 — a hero, from the values given	41s
04 — the agent writes its own checker	495s
05 — the loop runs	46s
What 04 produced	<code>check.mjs</code> , 264 lines — plus tests for it and a document explaining it, unasked
What it decided alone	to treat axe's <i>incomplete</i> results as failures

And here is what went wrong

Thirteen incidents while building this. **Nine were failures of the verifier or the harness, not of the page.**

WHAT

Nine gates ran against **somebody else's website** for a whole run
— 427 confident, correctly-formatted failures

axe passed a blank page. Zero violations, three breakpoints,
green

The loop hung for an hour on a stdin nobody closed

The **evidence harness fabricated** four clean iterations out of a
killed run

The gate was green and **the hero was illegible** — five of nine
pieces of text below the legal minimum

WHY IT IS IN THE DECK

A verifier that is confident and wrong is worse
than none

A measurement of nothing looks exactly like a
measurement of perfection

Third unbounded wait in one project

Absence read as success — fourth time, and the
only one that invented a table

axe does not fail text over a photograph. It marks
it *incomplete*

evidence/INCIDENTS.md — all thirteen, written down at the time

// 0:55 — 1:10 • SPRINT 2 RUNS IN THE BACKGROUND

The loop, and workflows

Start your loop now, then listen

Paste **prompt 05**. Then leave it alone — I will talk over it, and when it stops we will look at what it did.

```
Run npm run check. If it exits 0, stop and tell me.  
If it does not, read check-report.md, fix what it names, run it again.  
Repeat until it exits 0.  
  
NEVER change check.mjs to make a check pass.
```



On Claude Code you can make it structural instead of trusted:

```
/goal npm run check exits 0
```



The session cannot end until that command actually exits 0. Codex has no equivalent — there, the paragraph is the mechanism.

The whole loop is four sentences

Check first	Run the check <i>before</i> you fix anything. An agent asked to improve passing work will find a reason.
Hand back the report	Unchanged. The file a program wrote, not your summary of it.
Never touch the checker	The moment the thing being judged can edit the judge, every green result afterwards means nothing.
Stop on a program	Not when it feels finished. When something exits 0.

No script. Nothing to install. You can say this to any agent, today.

Why a fresh context beats a long one

ONE LONG CONVERSATION

Forty minutes of history.

Three abandoned approaches still in context.

The original brief, sixty thousand tokens ago.

Reasoning gets worse as it fills.

A FRESH CONTEXT EACH PASS

Reads `checks/report.md` — the current failures.

Reads `notes.md` — what was already tried, and what it cost.

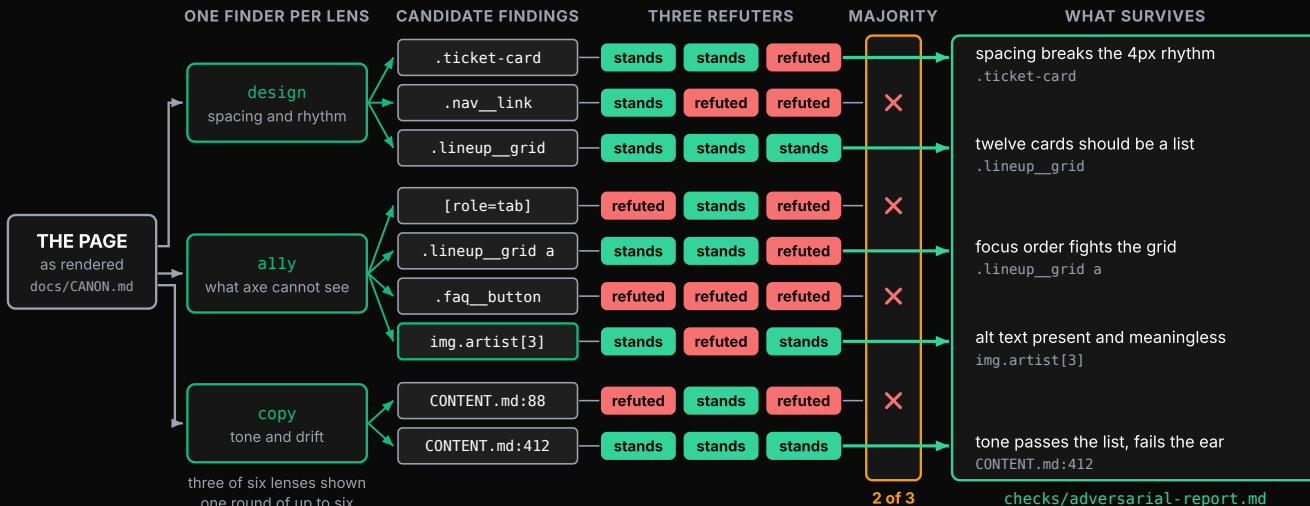
Reads the brief.

Notes on disk beat memory in context.

When one agent is not enough

One finder per lens, three refuters per finding

The work fans out, gets argued with, and only the part that survives a majority reaches a person.



Nothing here runs in lockstep.

Each candidate goes to three refuters whose only instruction is to argue that it is wrong, and reaches the filter when its own panel finishes. A lens that returns two findings does not hold up a lens that returns four.

The filter is arithmetic: two of three must stand. Every claim cites a selector or a file and line, or it is discarded unread.

Nothing on this page sets pass or fail. It opens items for a person to triage.

Adversarial review

The instruction is not "review this".

The instruction is **"refute this, and default to refuted if you are unsure"**.

```
const votes = await parallel(LENSES.map(lens => () =>
  agent(`Try to refute: ${finding}. Default to refuted if uncertain.`,
    { schema: VERDICT })))

const survives = votes.filter(v => !v.refuted).length >= 2
```



A reviewer asked to find problems finds problems. A reviewer asked to destroy a specific claim either destroys it or fails to.

Ship it

Deploy

```
npm run build  
npx netlify deploy --prod --dir=dist
```



It will ask you to sign in, then to pick or create a site. Take the URL it prints.

Put your URL on the board. We are going to look at all of them.

The gate was green. The hero was illegible.

Zero axe violations. Three breakpoints. Six page states. Twice, deterministically. Lighthouse accessibility 100.

Then I measured the hero by hand — screenshot the page with every glyph made transparent, sample the brightest patch behind each line, compute the real ratio.

TEXT	MEASURED
"Fourth edition"	2.94:1
"Ambient, techno and modular..."	3.02:1
"The Powerhouse, Hall E"	4.25:1
"Friday 12 – Sunday 14 June"	4.41:1

WHY AXE SAID NOTHING

It does not evaluate text over a background image. It does not fail it — it marks the pair **incomplete**, and in a zero-violations gate that is indistinguishable from correct.

Reading the CSS would not have found it either. The background there is a photograph, two scrim and a gradient composited together. No computed style says what colour that is.

So it became AC-61, and there are sixty-one criteria now instead of sixty.

What this does not do

Honesty

THE CLAIM

The a11y gate is green

The pixel diff passes

Adversarial review found nothing

All sixty-one criteria pass

THE LIMIT

Automated rules reach maybe 30–40% of real WCAG failures

It saw no change. It has no idea whether a change would be better

Models checking models. Least reliable tier, never the only one

Nothing here has an opinion about whether the design is good

Green means **no known defect**. It has never meant finished.

Where loops actually fail

It edits the test. The gate goes green and nothing was fixed. Forbid it in writing, and check that the file did not change.

It oscillates. Two wrong states, alternating. Usually two criteria that contradict each other. Cap the iterations and read the log.

It keeps going after it is done. Run the verifier *first*, every pass.

It runs out of context mid-task. Put the state in a file.

It reports success without running anything. Only believe the exit code.

It hangs. Not failing, just waiting. Every step needs a deadline — this repository lost six minutes to exactly that, and it is written down.

Take this to work on Monday

Do not start with a whole landing page. Start with one gate.

1. Pick the check your team already does by hand and resents.
2. Make it a program that exits 0 or 1.
3. Put the failure output somewhere an agent can read.
4. Only then put an agent in front of it.

The loop is the easy part. The oracle is the work.

// EVERYTHING FROM TODAY

TURBINE

`github.com/hculap/waysconf-2026-agentic-loops`

The brief · every prompt I used · the verifier · the Figma plugin ·
the evidence from both clean-room runs · these slides

Questions. And then go and put your URL on the board.